Desarrollo de un Sistema Híbrido de Detección y Mitigación de Alucinaciones en LLMs mediante Cuantificación de Incertidumbre y Arquitecturas RAG Agénticas

1. Introducción: El Desafío de la Fiabilidad en la Generación de Lenguaje Natural

La proliferación de Grandes Modelos de Lenguaje (LLMs, por sus siglas en inglés) ha transformado radicalmente el panorama de la inteligencia artificial, permitiendo avances sin precedentes en tareas de razonamiento, resumen y generación de código. Sin embargo, la adopción de estas tecnologías en entornos críticos —como la medicina, el derecho y la toma de decisiones corporativas— se ve obstaculizada por un fenómeno persistente y peligroso: la alucinación.[1] Definida técnicamente como la generación de contenido que es gramaticalmente plausible y semánticamente coherente, pero factualmente incorrecto o no fundamentado en las fuentes proporcionadas, la alucinación representa la manifestación de la brecha entre la capacidad lingüística del modelo y su fiabilidad factual.

En el contexto de la investigación para un Trabajo de Fin de Máster (TFM), el abordaje de este problema requiere una distinción técnica rigurosa entre dos dimensiones de la fiabilidad: la conciencia interna del modelo sobre su propia ignorancia (incertidumbre epistémica) y la precisión verificable de sus afirmaciones frente a una realidad externa. La literatura científica reciente, particularmente los trabajos de Kuhn et al. (2023), Farquhar et al. (2024), y Min et al. (2023), sugiere que no existe una única "bala de plata" para resolver las alucinaciones. Por el contrario, la solución reside en sistemas híbridos que integren métricas de detección basadas en la incertidumbre semántica con arquitecturas de mitigación activa, específicamente aquellas que emplean Generación Aumentada por Recuperación (RAG) con capacidades agénticas y reflexivas, como Self-RAG y Corrective-RAG (CRAG).[3]

Este informe técnico despliega una propuesta exhaustiva para el desarrollo de tal sistema. A diferencia de los enfoques convencionales que tratan la alucinación como un error monolítico, aquí se descompone el problema en métricas operacionales: la **Probabilidad de Invención ($P_{hallucination}$)**, fundamentada en la Entropía Semántica, y el **Porcentaje de Veracidad ($FactScore$ion)**, basado en la descomposición atómica y verificación externa. A continuación, se detalla la fundamentación matemática de estas métricas y la arquitectura de ingeniería de software necesaria para implementar un flujo de trabajo agéntico que no solo

detecte el error, sino que lo corrija autónomamente mediante estrategias de recuperación avanzadas.

---

2. Fundamentación Matemática y Algorítmica de las Métricas de Detección

La evaluación de la fiabilidad en LLMs ha evolucionado desde métricas de similitud de n-gramas (como ROUGE o BLEU), que son insuficientes para medir la veracidad factual, hacia métricas basadas en modelos y en la teoría de la información. Para este TFM, nos centramos en dos pilares métricos que abordan la alucinación desde perspectivas complementarias: interna y externa.

2.1. Probabilidad de Invención ($P_{hallucination}$) basada en Entropía Semántica

La premisa subyacente a la **Probabilidad de Invención** es que las alucinaciones están correlacionadas positivamente con la incertidumbre epistémica del modelo. Si un modelo "sabe" que no sabe la respuesta, o si confabula, su distribución de probabilidad sobre las posibles respuestas debería reflejar indecisión. Sin embargo, calcular esta incertidumbre en la generación de lenguaje natural (NLG) presenta un desafío matemático único: la **invariancia semántica**.

2.1.1. El Problema de la Entropía Léxica en NLG

En tareas de clasificación tradicionales, la incertidumbre se mide mediante la entropía de Shannon sobre las clases de salida. En NLG, la salida es una secuencia de tokens. Si calculamos la entropía directamente sobre las secuencias de tokens (entropía léxica), obtenemos una medida espuria de incertidumbre. Esto se debe a que un mismo significado puede expresarse mediante múltiples secuencias de palabras diferentes.[6]

Por ejemplo, ante la pregunta "¿Cuál es la capital de Francia?", el modelo podría asignar alta probabilidad a las secuencias "París", "Es París", "La capital es París". Aunque el significado es idéntico y el modelo está seguro del concepto, la entropía léxica sería alta debido a la variabilidad en la redacción. Esto llevaría a un falso positivo en la detección de alucinaciones: el modelo parece incierto, pero solo es "léxicamente diverso".[8]

2.1.2. Definición Formal de Entropía Semántica (SE)

Para corregir esto, Kuhn et al. (2023) y Farquhar et al. (2024) introducen la **Entropía Semántica (SE)**. La idea central es calcular la entropía sobre el espacio de *significados* en lugar del espacio de *secuencias*.

Sea $\mathcal{X}$ el espacio de entradas (prompts) y $\mathcal{V}^*$ el espacio de todas las secuencias posibles de tokens sobre un vocabulario $\mathcal{V}$. Dada una entrada $x \in \mathcal{X}$, el modelo genera una secuencia $s \in \mathcal{V}^*$ con probabilidad $p(s|x)$.

Definimos una relación de equivalencia semántica $E(s, s')$ que es verdadera si y solo si las secuencias s y s' comparten el mismo significado en el contexto de x. Esta relación de equivalencia induce una partición del espacio de secuencias en un conjunto de **Clases de Equivalencia Semántica** $C(x) = \{c_1, c_2, \dots, c_K\}$, donde cada clase c_k representa un significado único.[5]

La probabilidad de un significado (o clase) c se define marginalizando sobre todas las secuencias que pertenecen a esa clase:

$$p(c | x) = \sum_{s \in c} p(s | x)$$
La Entropía Semántica se define entonces como la entropía de la distribución sobre estas clases de significado:

$$SE(x) = H(C|x) = - \sum_{c \in C} p(c | x) \log p(c | x)$$
Sustituyendo la probabilidad de la clase:

$$SE(x) = - \sum_{c \in C} \left(\sum_{s \in c} p(s | x) \right) \log \left(\sum_{s \in c} p(s | x) \right)$$
Esta formulación asegura que si el modelo distribuye su masa de probabilidad entre muchas frases diferentes que significan lo mismo (ej. "París", "Es París"), la entropía semántica será baja (cercana a cero), indicando certeza conceptual. Por el contrario, si la masa de probabilidad se divide entre significados contradictorios (ej. "París", "Londres", "Berlín"), la entropía semántica será alta, señalando una alta **Probabilidad de Invención**.[10]

2.1.3. Algoritmo de Estimación: Muestreo y Clustering por Implicación Bidireccional

Dado que el espacio $\mathcal{V}^*$ es infinito, el cálculo exacto de la suma es intratable. Se utiliza un estimador de Monte Carlo en tres etapas. Este algoritmo es el núcleo técnico de la implementación de la métrica $P_{hallucination}$ en el sistema híbrido propuesto.

Etapa 1: Generación Estocástica (Sampling)
Se generan M secuencias candidatas independientes $\{s^{(1)}, s^{(2)}, \dots, s^{(M)}\}$ condicionadas a la entrada x, utilizando un esquema de muestreo con temperatura (típicamente $T \approx 0.7$ a 1.0 para fomentar diversidad).10
Etapa 2: Agrupamiento Semántico (Clustering)

El desafío crítico es determinar algorítmicamente si $s^{(i)}$ y $s^{(j)}$ pertenecen a la misma clase c. Para ello, se emplea el criterio de Implicación Bidireccional (Bidirectional Entailment) utilizando un modelo de Inferencia de Lenguaje Natural (NLI) o un LLM instruido.11
La condición para agrupar $s^{(i)}$ y $s^{(j)}$ es:

$$s^{(i)} \equiv s^{(j)} \iff (s^{(i)} \Rightarrow s^{(j)}) \land (s^{(j)} \Rightarrow s^{(i)})$$

Donde $\Rightarrow$ denota implicación lógica. Si la secuencia i implica la j y viceversa, son semánticamente equivalentes.
Algoritmo de Clustering (Greedy):

1. Inicializar el conjunto de clusters $C = \emptyset$.
2. Para cada secuencia generada $s^{(m)}$:
   a. Comparar $s^{(m)}$ con un representante de cada cluster existente $c_k \in C$.
   b. Verificar implicación bidireccional mediante un modelo NLI (ej. DeBERTa-Large-MNLI o GPT-3.5/4).
   c. Si hay implicación bidireccional con el cluster c_k, añadir $s^{(m)}$ a c_k.
   d. Si no coincide con ningún cluster, crear un nuevo cluster $c_{new} = \{s^{(m)}\}$ y añadirlo a C.12

Prompt de Verificación (para implementación con LLM):
Para implementar esto con un LLM, se utiliza el siguiente template validado en la literatura:

> "We are evaluating answers to the question: {question}. Here are two possible answers: Possible Answer 1: {text1}. Possible Answer 2: {text2}. Does Possible Answer 1 semantically entail Possible Answer 2? Respond with entailment, contradiction, or neutral." [12]

Etapa 3: Estimación de la Entropía
Una vez obtenidos los clusters, se aproxima la probabilidad de cada clase. Existen dos estimadores principales:

1. **Estimador Discreto (DSE):** Simplemente cuenta la frecuencia de muestras en cada cluster. Es adecuado para modelos "Caja Negra" (API) donde no tenemos acceso a los logits.
   $$p(c_k | x) \approx \frac{|c_k|}{M} $$ $$ SE(x) \approx - \sum_{k} \frac{|c_k|}{M} \log \frac{|c_k|}{M}$$
2. **Estimador Log-Prob (Integración de Logits):** Si se tiene acceso a las probabilidades de los tokens (White-box), se suman las probabilidades normalizadas de las secuencias dentro del cluster, lo que proporciona una estimación más robusta con menos muestras.[5]

La **Probabilidad de Invención** se define entonces como una función monótona creciente de $SE(x)$. En la práctica, se establece un umbral τ; si $SE(x) > \tau$, se considera que el modelo está alucinando o confabulando.[10]

2.2. Porcentaje de Veracidad ($FactScore$): Verificación Externa

Mientras que la Entropía Semántica detecta la inconsistencia interna, el sistema requiere una métrica para validar la precisión factual objetiva de las respuestas generadas, especialmente en contextos de RAG donde la adherencia a la fuente es crítica. Min et al. (2023) propusieron **FActScore** (Fine-grained Atomic Evaluation of Factual Precision) para superar las limitaciones de las evaluaciones binarias a nivel de documento.[13]

2.2.1. El Concepto de Hecho Atómico

La evaluación de textos largos es compleja porque una generación puede contener una mezcla de verdades, errores y opiniones. FActScore resuelve esto descomponiendo el texto en **Hechos Atómicos**.

Un hecho atómico se define como una unidad proposicional indivisible de información que puede ser verificada independientemente como verdadera o falsa con respecto a una fuente de conocimiento $\mathcal{K}$ (ej. Wikipedia o un corpus corporativo). Matemáticamente, una generación y se modela como un conjunto de proposiciones lógicas $A_y = \{a_1, a_2, \dots, a_n\}$.[15]

2.2.2. Formulación Matemática de la Métrica

El FActScore se define formalmente como la precisión (precision) de los hechos atómicos generados. No penaliza por exhaustividad (recall), sino por corrección.

Sea y la respuesta generada por el modelo M ante el prompt x. Sea A_y el conjunto de hechos atómicos extraídos de y. Definimos una función indicadora de soporte $\mathbb{I}(a, \mathcal{K})$:

$$\mathbb{I}(a, \mathcal{K}) = \begin{cases} 1 & \text{si } a \text{ está soportado por } \mathcal{K} \\ 0 & \text{en caso contrario} \end{cases}$$
El FActScore para una generación individual y se calcula como el promedio de soporte:

$$FActScore(y) = \frac{1}{|A_y|} \sum_{a \in A_y} \mathbb{I}(a, \mathcal{K}) \times 100\%$$
Para evaluar el modelo M en general sobre un conjunto de pruebas $\mathcal{D}$, el FActScore es la esperanza matemática sobre las generaciones:

$$FActScore(M) = \mathbb{E}_{x \sim \mathcal{D}} \left[\frac{1}{|A_{M(x)}|} \sum_{a \in A_{M(x)}} \mathbb{I}(a, \mathcal{K}) \right]$$
[14]

2.2.3. Pipeline de Implementación: "Decompose-Then-Verify"

Para implementar el cálculo de esta métrica en el TFM, se debe orquestar un pipeline de tres etapas conocido como "Decompose-Then-Verify".

Etapa 1: Generación de Hechos Atómicos (Atomic Fact Generation)
Se utiliza un LLM instructor (como GPT-4 o Llama-3-Instruct) para segmentar la respuesta del modelo objetivo. El prompt es crucial aquí para asegurar que las oraciones sean autocontenidas.

- *Estrategia:* Segmentar por oraciones y luego simplificar proposiciones complejas. Por ejemplo, "Obama nació en Hawái y fue presidente" se descompone en "Obama nació en Hawái" y "Obama fue presidente".
- *Referencia Técnica:* Min et al. (2023) proporcionan modelos finetuneados para esta tarea, pero en un TFM se puede usar *few-shot prompting*.[16]

Etapa 2: Recuperación de Conocimiento (Knowledge Retrieval)
Para cada hecho atómico a_i, el sistema debe buscar evidencia en $\mathcal{K}$. A diferencia del RAG estándar que busca para responder una pregunta, aquí la "query" es el hecho atómico a_i. Se utiliza un recuperador denso (ej. Contriever o embeddings de OpenAI) para obtener un conjunto de pasajes de evidencia $D_{evidence} = \{d_1, \dots, d_k\}$.14
Etapa 3: Verificación de Hechos (Fact Validation)
Un modelo "Juez" evalúa si $D_{evidence}$ soporta a_i. La validación se formula como un problema de implicación textual o clasificación binaria:

$$\text{Label} = \text{Model}_{eval}(a_i, D_{evidence}) \in \{\text{Soportado}, \text{No Soportado}\}$$

Si el modelo evaluador asigna una probabilidad de "Soportado" superior a 0.5 (o devuelve "True"), entonces $\mathbb{I}(a_i, \mathcal{K}) = 1$.
Este enfoque modular permite diagnosticar exactamente qué parte de la respuesta es alucinada, proporcionando una granularidad que métricas globales no ofrecen.[18]

---

3. Arquitecturas Avanzadas de Mitigación: RAG Agéntico

Una vez establecidas las métricas para detectar y cuantificar la alucinación, el siguiente componente del TFM es el sistema de mitigación. La literatura actual ha superado el paradigma de RAG estático (donde simplemente se recupera y se concatena) hacia arquitecturas "agénticas" que razonan sobre la calidad de la recuperación y la necesidad de buscar información. Analizamos dos arquitecturas estado del arte: **Self-RAG** y **CRAG**, y cómo

se estructuran técnicamente.

3.1. Self-RAG: Recuperación y Reflexión Autónoma

Self-RAG (Self-Reflective Retrieval-Augmented Generation), propuesto por Asai et al. (2024), introduce la idea de que el LLM no solo debe generar texto, sino también generar sus propias señales de control mediante **Tokens de Reflexión**. Este enfoque permite al modelo criticar su propia salida y decidir dinámicamente si necesita buscar información externa.[3]

3.1.1. Taxonomía de Tokens de Reflexión

Self-RAG entrena al modelo para emitir tokens especiales durante la inferencia. Estos tokens funcionan como disparadores de lógica de control en el sistema.

Tipo de Token	Nombre	Definición y Función	Valores Posibles
Recuperación	Retrieve	Decide si es necesario buscar información externa para el contexto actual.	{yes, no, continue}
Relevancia	IsRel	Evalúa si el pasaje recuperado d es relevante para la consulta x.	{relevant, irrelevant}
Soporte	IsSup	Verifica si la respuesta generada y está fundamentada en el pasaje d.	{fully, partially, no support}
Utilidad	IsUse	Evalúa la calidad y utilidad general de la respuesta.	{1, 2, 3, 4, 5}

3

3.1.2. Algoritmo de Inferencia Agéntica

La implementación de Self-RAG no es un simple paso *feed-forward*, sino un algoritmo de

ramificación y poda (branching and pruning) a nivel de segmento.

1. Predicción de Necesidad (Retrieve):
   Dado el prompt x y la generación previa $y_{<t}$, el modelo predice el token Retrieve.
   - Si Retrieve == no, el modelo genera el siguiente segmento utilizando su conocimiento paramétrico.
   - Si Retrieve == yes, el sistema pausa la generación y activa el módulo de recuperación $\mathcal{R}$ para obtener documentos D.[19]
2. Generación Paralela y Crítica:
   Para cada documento recuperado $d_i \in D$, el modelo genera una respuesta candidata $y_{t}^{(i)}$ y simultáneamente predice los tokens de crítica:
   - Evalúa la relevancia del documento (IsRel).
   - Evalúa si la respuesta candidata se deriva del documento (IsSup).
   - Evalúa la utilidad final (IsUse).
3. Ranking y Selección (Self-Correction):
   El sistema calcula un puntaje compuesto para cada candidato $y_{t}^{(i)}$ basado en una suma ponderada de las probabilidades de los tokens de reflexión.
   $$Score(y_t^{(i)}) = w_1 P(\text{IsRel}) + w_2 P(\text{IsSup}) + w_3 P(\text{IsUse})$$
   Se selecciona el segmento con el puntaje más alto (segment-level beam search). Esto asegura que el sistema prefiera respuestas que no solo sean útiles, sino que estén factualmente soportadas por evidencia relevante, mitigando directamente la alucinación.[19]

3.2. CRAG: Generación Aumentada por Recuperación Correctiva

Mientras que Self-RAG integra la evaluación dentro del LLM generador, **Corrective-RAG (CRAG)**, propuesto por Yan et al. (2024), externaliza la validación a un componente especializado ligero: el **Evaluador de Recuperación**. CRAG se enfoca en la robustez frente a recuperaciones defectuosas, una causa común de alucinaciones en RAG estándar.[4]

3.2.1. El Evaluador de Recuperación (Retrieval Evaluator)

El núcleo de CRAG es un modelo clasificador (típicamente T5-Large fine-tuned) que evalúa la calidad de los documentos recuperados antes de que lleguen al LLM generador. Este evaluador asigna un grado de confianza a la recuperación y dispara tres posibles acciones correctivas.[21]

3.2.2. Acciones y Mecanismos de Disparo

El flujo de control de CRAG se basa en los grados de confianza:

1. **Acción CORRECTA (High Confidence):**
   - *Condición:* El evaluador determina que los documentos son relevantes.
   - *Mecanismo:* Se ejecuta un algoritmo de **Refinamiento de Conocimiento** (ver 3.2.3). No se realiza búsqueda adicional.

2. **Acción INCORRECTA (Low Confidence):**
   - *Condición:* Los documentos recuperados son irrelevantes o engañosos.
   - *Mecanismo:* Se descartan los documentos recuperados del vector store. El sistema activa una **Búsqueda Web** (Web Search) para buscar información fresca y corregir la falta de datos en el corpus estático.[23]
3. **Acción AMBIGUA (Ambiguous):**
   - *Condición:* Incertidumbre sobre la relevancia.
   - *Mecanismo:* Enfoque híbrido. Se procesan los documentos recuperados *y* se complementan con búsqueda web para asegurar cobertura.[21]

3.2.3. Algoritmo "Decompose-Then-Recompose"

Una innovación técnica crucial de CRAG para mejorar el FActScore es el algoritmo de refinamiento de documentos. Incluso un documento relevante puede contener "ruido" que confunda al LLM.

- **Decompose (Descomposición):** El documento recuperado se segmenta en unidades finas (ej. oraciones o fragmentos de proposiciones).
- **Filter (Filtrado):** El evaluador califica cada fragmento individualmente. Se eliminan los fragmentos irrelevantes.
- Recompose (Recomposición): Se concatenan únicamente los fragmentos de alta relevancia para formar un contexto depurado y denso.
  Este proceso maximiza la relación señal/ruido en el prompt del LLM, reduciendo significativamente la probabilidad de alucinaciones inducidas por contexto irrelevante.25

---

4. Diseño del Sistema Híbrido: Implementación Técnica para el TFM

Basado en el análisis de las métricas y las arquitecturas, se propone para el TFM un sistema híbrido que utiliza la **Entropía Semántica como mecanismo de activación (Gating)** y una arquitectura de **Grafo Agéntico (LangGraph)** que integra la lógica de CRAG y Self-RAG.

4.1. Lógica de Integración Híbrida

La integración propuesta responde a la necesidad de eficiencia y precisión. No todas las consultas requieren el costo computacional de un pipeline agéntico completo.

1. Detección Preventiva (Fase de Incertidumbre):
   El sistema recibe la consulta y realiza una generación preliminar rápida (sampling). Calcula la Entropía Semántica ($P_{hallucination}$).
   - Si $P_{hallucination} < \tau$ (Umbral bajo): El modelo tiene alta certeza interna. Se entrega la respuesta directa (Fast Path).
   - Si $P_{hallucination} \ge \tau$ (Alta incertidumbre): Se diagnostica riesgo de

invención. Se activa el **Modo Agéntico de Mitigación** (Slow Path).[5]

2. Mitigación Agéntica (Fase RAG Híbrido):
   Una vez activado, el sistema emplea la lógica de CRAG para garantizar la calidad de los datos (Búsqueda Web + Refinamiento) y, finalmente, utiliza la lógica de Self-RAG (Tokens de Soporte) durante la generación final para asegurar que la respuesta se adhiera estrictamente al contexto depurado.

4.2. Estructura de Implementación con LangGraph

Para implementar este flujo de trabajo complejo, se recomienda utilizar **LangGraph**, una extensión de LangChain diseñada para flujos cíclicos y agénticos.[28] A continuación se detalla la estructura de grafos (nodos y aristas) requerida.

4.2.1. Definición del Estado del Grafo (GraphState)

El estado es la estructura de datos que persiste y muta a través de los nodos del grafo.

Python

```python
class GraphState(TypedDict):
    question: str                # Consulta del usuario
    generation: str              # Respuesta actual
    documents: List[str]         # Contexto recuperado
    uncertainty_score: float     # Entropía Semántica calculada
    hallucination_risk: bool     # Flag de activación
    web_search_needed: bool      # Decisión de CRAG
```

4.2.2. Nodos del Sistema (Functions)

1. **Node: compute_uncertainty**
   - *Input:* question.
   - *Proceso:* Genera 5 muestras con temperatura 1.0. Ejecuta clustering bidireccional (usando un modelo NLI ligero o API call). Calcula SE.
   - *Output:* Actualiza uncertainty_score y hallucination_risk.
2. **Node: retrieve_documents**
   - *Input:* question.
   - *Proceso:* Consulta al Vector Store (ej. ChromaDB indexado con documentos del dominio).
   - *Output:* documents.
3. **Node: grade_retrieval_crag (El Evaluador)**
   - *Input:* question, documents.

- *Proceso:* Utiliza un LLM (o T5) con un prompt de evaluación para clasificar la relevancia de cada documento. Aplica lógica CRAG: si la calidad es baja, marca web_search_needed = True. Si es alta, aplica decompose_and_recompose.[29]
- *Prompt Técnico: "Grade the relevance of the following document to the user query. Give a binary score 'yes' or 'no'..."*

4. **Node: web_search_tool**
   - *Input:* question.
   - *Proceso:* Ejecuta búsqueda (ej. Tavily API) si CRAG lo solicita.
   - *Output:* Añade resultados a documents.
5. **Node: generate_self_rag**
   - *Input:* question, documents (refinados).
   - *Proceso:* Genera la respuesta final. En esta etapa, se puede simular Self-RAG mediante prompting si no se tiene acceso a los pesos del modelo original, instruyendo al modelo a citar y verificar: *"Generate a response based strictly on the context. If the context does not support the answer, state it."* (Simulación de IsSup).

4.2.3. Flujo de Aristas (Edges)

- **Conditional Edge 1 (check_risk):**
  - Desde compute_uncertainty.
  - Si hallucination_risk == True $\rightarrow$ Ir a retrieve_documents.
  - Si hallucination_risk == False $\rightarrow$ Ir a END (Devolver generación inicial).
- **Conditional Edge 2 (decide_source):**
  - Desde grade_retrieval_crag.
  - Si web_search_needed == True $\rightarrow$ Ir a web_search_tool.
  - Si web_search_needed == False $\rightarrow$ Ir a generate_self_rag.

4.3. Comparativa y Valor Científico del Enfoque

Este diseño supera las limitaciones de los sistemas RAG tradicionales al introducir "metacognición" (vía Entropía Semántica) y mecanismos correctivos activos.

Característica	**RAG Tradicional**	**Sistema Híbrido Propuesto (TFM)**
Trigger de Recuperación	Siempre recupera (costoso/ruidoso)	Dinámico basado en Incertidumbre ($P_{hallucination}$)
Calidad de Contexto	Top-K documentos crudos	Filtrado y Refinado (CRAG Decompose-Recompose)

Verificación de Respuesta	Inexistente	Intrínseca (Self-RAG Tokens) y Extrínseca (FActScore)
Robustez	Susceptible a documentos irrelevantes	Búsqueda Web correctiva ante fallos de corpus

27

---

5. Conclusiones y Próximos Pasos

El desarrollo de un Sistema Híbrido de Detección y Mitigación de Alucinaciones representa un avance significativo hacia la IA confiable. A través de la formalización matemática de la **Entropía Semántica**, logramos dotar al modelo de la capacidad de introspección para detectar cuándo está confabulando ($P_{hallucination}$). Complementariamente, la métrica **FActScore** proporciona un marco riguroso para la auditoría factual externa.

La arquitectura de implementación propuesta, orquestada mediante **LangGraph** y basada en los paradigmas de **Self-RAG** y **CRAG**, ofrece una solución técnica robusta. Al integrar la evaluación de recuperación (CRAG) con la generación reflexiva (Self-RAG), el sistema no solo mitiga la alucinación, sino que mejora activamente la calidad de la información presentada al usuario. Para el TFM, la ejecución de este diseño permitirá demostrar empíricamente cómo la combinación de incertidumbre epistémica y agentes correctivos supera a los modelos base en tareas de alta precisión factual.

Referencias Bibliográficas

Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. *International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.2310.11511

Farquhar, S., Kossen, J., Kuhn, L., & Gal, Y. (2024). Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017), 625–630. https://doi.org/10.1038/s41586-024-07421-0

Kuhn, L., Gal, Y., & Farquhar, S. (2023). Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation. *International Conference on Learning Representations (ICLR)*. https://doi.org/10.48550/arXiv.2302.09664

Min, S., Krishna, K., Lyu, X., Lewis, M., Yih, W., Koh, P., Iyyer, M., Zettlemoyer, L., &

Hajishirzi, H. (2023). FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://doi.org/10.18653/v1/2023.emnlp-main.741

Yan, S.-Q., Gu, J.-C., Zhu, Y., & Ling, Z.-H. (2024). Corrective Retrieval Augmented Generation. *arXiv preprint arXiv:2401.15884*. https://doi.org/10.48550/arXiv.2401.15884

Obras citadas

1. Detecting hallucinations in large language models using semantic entropy - PubMed, fecha de acceso: diciembre 11, 2025, https://pubmed.ncbi.nlm.nih.gov/38898292/
2. Detecting hallucinations in large language models using semantic entropy - ResearchGate, fecha de acceso: diciembre 11, 2025, https://www.researchgate.net/publication/381550138_Detecting_hallucinations_in_large_language_models_using_semantic_entropy
3. The Self-RAG Shortcut Every AI Expert Wishes They Knew - ProjectPro, fecha de acceso: diciembre 11, 2025, https://www.projectpro.io/article/self-rag/1176
4. Corrective RAG (CRAG): Workflow, implementation, and more - Meilisearch, fecha de acceso: diciembre 11, 2025, https://www.meilisearch.com/blog/corrective-rag
5. Beyond Semantic Entropy: Boosting LLM Uncertainty Quantification with Pairwise Semantic Similarity - ACL Anthology, fecha de acceso: diciembre 11, 2025, https://aclanthology.org/2025.findings-acl.234.pdf
6. LINGUISTIC INVARIANCES FOR UNCERTAINTY ESTIMATION IN NATURAL LANGUAGE GENERATION - OpenReview, fecha de acceso: diciembre 11, 2025, https://openreview.net/pdf?id=VD-AYtP0dve
7. Kernel Language Entropy: Fine-grained Uncertainty Quantification for LLMs from Semantic Similarities - arXiv, fecha de acceso: diciembre 11, 2025, https://arxiv.org/html/2405.20003v1
8. [2302.09664] Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation in Natural Language Generation - arXiv, fecha de acceso: diciembre 11, 2025, https://arxiv.org/abs/2302.09664
9. arXiv:2302.09664v3 [cs.CL] 15 Apr 2023, fecha de acceso: diciembre 11, 2025, https://arxiv.org/pdf/2302.09664
10. Detecting LLM confabulations with semantic entropy | Schrier's Sudelbücher, fecha de acceso: diciembre 11, 2025, https://jschrier.github.io/blog/2024/07/31/Detecting-LLM-confabulations-with-semantic-entropy.html
11. Semantic Entropy Probes: Robust and Cheap Hallucination Detection in LLMs - arXiv, fecha de acceso: diciembre 11, 2025, https://arxiv.org/html/2406.15927v1
12. lorenzkuhn/semantic_uncertainty - GitHub, fecha de acceso: diciembre 11, 2025, https://github.com/lorenzkuhn/semantic_uncertainty
13. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation - ACL Anthology, fecha de acceso: diciembre 11, 2025,

https://aclanthology.org/2023.emnlp-main.741/
14. FActScore: Metric for Factual Precision in LLMs - Emergent Mind, fecha de acceso: diciembre 11, 2025, https://www.emergentmind.com/topics/factscore
15. [2305.14251] FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation - arXiv, fecha de acceso: diciembre 11, 2025, https://arxiv.org/abs/2305.14251
16. A Closer Look at Claim Decomposition - arXiv, fecha de acceso: diciembre 11, 2025, https://arxiv.org/html/2403.11903v1
17. A package to evaluate factuality of long-form generation. Original implementation of our EMNLP 2023 paper "FActScore - GitHub, fecha de acceso: diciembre 11, 2025, https://github.com/shmsw25/FActScore
18. OpenFActScore: Open-Source Atomic Evaluation of Factuality in Text Generation - arXiv, fecha de acceso: diciembre 11, 2025, https://arxiv.org/html/2507.05965v1
19. Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection - arXiv, fecha de acceso: diciembre 11, 2025, https://arxiv.org/abs/2310.11511
20. This includes the original implementation of SELF-RAG: Learning to Retrieve, Generate and Critique through self-reflection by Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. - GitHub, fecha de acceso: diciembre 11, 2025, https://github.com/AkariAsai/self-rag
21. [2401.15884] Corrective Retrieval Augmented Generation - arXiv, fecha de acceso: diciembre 11, 2025, https://arxiv.org/abs/2401.15884
22. Corrective RAG (CRAG) in Action - Analytics Vidhya, fecha de acceso: diciembre 11, 2025, https://www.analyticsvidhya.com/blog/2024/12/corrective-rag/
23. Corrective Retrieval-Augmented Generation for More Reliable RAG Systems - Kore.ai, fecha de acceso: diciembre 11, 2025, https://www.kore.ai/blog/corrective-rag-crag
24. Corrective RAG: Enhancing Large Language Model Outputs through Self-Verification | by Pankaj Tiwari | Accredian | Medium, fecha de acceso: diciembre 11, 2025, https://medium.com/accredian/corrective-rag-enhancing-large-language-model-outputs-through-self-verification-6e43dd2beb9f
25. Corrective Retrieval Augmented Generation - arXiv, fecha de acceso: diciembre 11, 2025, https://arxiv.org/html/2401.15884v2
26. CRAG (Corrective Retrieval-Augmented Generation) in LLM: What It Is and How It Works | by Sahin Ahmed, Data Scientist | Medium, fecha de acceso: diciembre 11, 2025, https://medium.com/@sahin.samia/crag-corrective-retrieval-augmented-generation-in-llm-what-it-is-and-how-it-works-ce24db3343a7
27. Hallucination Mitigation for Retrieval-Augmented Large Language Models: A Review - MDPI, fecha de acceso: diciembre 11, 2025, https://www.mdpi.com/2227-7390/13/5/856
28. Self-Rag: A Guide With LangGraph Implementation | DataCamp, fecha de acceso: diciembre 11, 2025, https://www.datacamp.com/tutorial/self-rag
29. Corrective RAG (CRAG) - GitHub Pages, fecha de acceso: diciembre 11, 2025, https://langchain-ai.github.io/langgraph/tutorials/rag/langgraph_crag/

30. Corrective RAG (CRAG) Implementation With LangGraph - DataCamp, fecha de acceso: diciembre 11, 2025, https://www.datacamp.com/tutorial/corrective-rag-crag
31. RAG Hallucinations Explained: Causes, Risks, and Fixes - Mindee, fecha de acceso: diciembre 11, 2025, https://www.mindee.com/blog/rag-hallucinations-explained